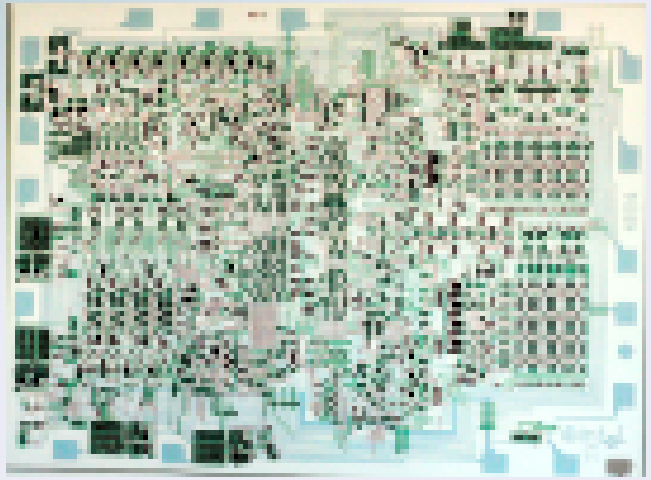


Lecture 14a: Enhancing Performance by Understanding Computer Architecture

CSCI11: **Computer Architecture and Organization**

Remaining Schedule

Day	Content
5/4	Enhancing Performance by Understanding Computer Architecture
5/6	Work on Final Project and 1:1 Discussions
5/11	**Review for Final Exam** and 1:1 Discussions
5/13	Work on Final Project and 1:1 Discussions
5/18	Work on Final Project and 1:1 Discussions
5/20	**Final Exam – Wed, May 21, 2025 6-8:30 PM**

SMC 1:1 Review

1. Your SMC will be submitted as a separate Github repo. I recommend that it is private and I am added as a collaborator, lkoepsel@wellys.com . This is the same way your assignment repo is setup.
2. **I will need to be added prior to your scheduled time. Do not wait, do not forget.**
3. In the 1:1, I'll review the repo with you including copying your final SMC code and running it in LC3Tools. The name of your **SMC** code will be *smc.asm* and it will be in the *code* folder.
4. Be sure to continue to review the SMC guidance at the top of the Instructor repo.
5. I'll score the Final Projects, once I've gone through all of the 1:1's.

Enhancing Performance by Understanding Computer Architecture

- Native vs. Non-native (C++/Rust vs. Java)
- Compiler Optimizations
- Reducing System Calls
- Enhancing Performance using SIMD
- Bigger, Better, Faster Chip

Note: Much of this presentation is verbatim from the website referenced. I'm not an expert on Javascript, x86 assembly, Rust etc. Thats not the point, the point is to see examples of what we reviewed in class in current technical discussions.

Native vs. Non-native (C++ or Rust vs. Java)

- When discussing performance enhancements, we're talking almost exclusively about *native code* as in **machine code** or the **ISA of the CPU**.
- For example, Java while is compiled, it compiles to byte-code of the JVM (Java Virtual Machine), this can be optimized to an extent, however, if you are pursuing performance...only native code will do
- And yes, "*GraalVM analyzes your Java application and its dependencies during build time, then compiles the bytecode directly to platform-specific machine code (e.g., Windows .exe, Linux binary, macOS app)*"
- However, that is an exception, not the rule as it requires giving up the ubiquity of Java, *write once, run anywhere*.

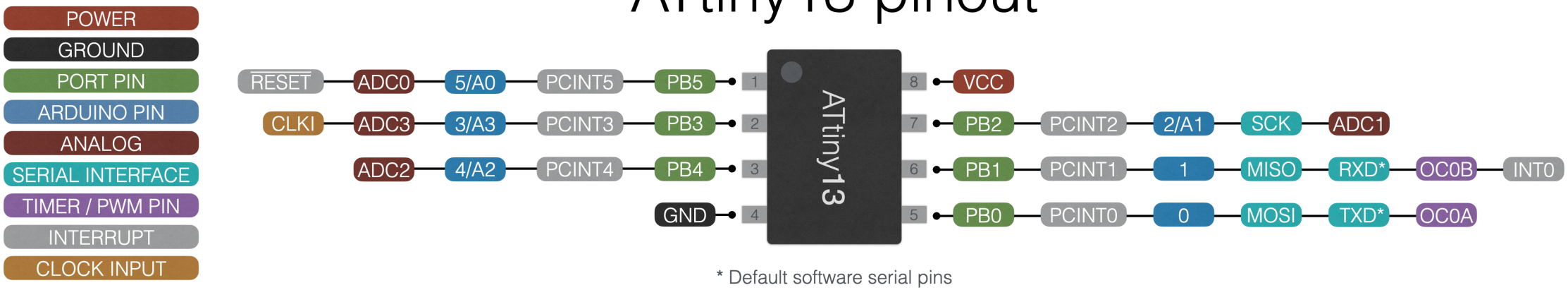
Compiler Optimizations

Microchip AVR family (*ATmegaxxx* or *ATtinyxxx*)

Chip	Flash	EEPROM	SRAM	ADC	Clock/Timers	Cost (DIP)
ATmega328P	32KB	1KB	2KB	8	3	$2.10
ATtiny13A	1KB	64	64	4	1	$.90

- The ATmega328P is the Arduino Uno R3, used in the 10C class
- I use the ATtiny13A for very small embedded projects and with my goal of using for this class

ATtiny13 pinout



* Default software serial pins

http://github.com/MCUdude/MicroCore

"*Optimize Your C Code for 8-bit AVR Microcontrollers*", Microchip

- AVR uses Harvard architecture – with separate memories and buses for program and data
- Fast-access register file of 32 × 8 general purpose working registers with a single clock cycle access time
- In one clock cycle, AVR can feed two arbitrary registers from the register file to the ALU, perform an operation, and write back the result to the register file

Data Types and Sizes

Using `#include <stdint.h>`

Use the smallest applicable data type as possible.

Data Type	Name	Size
signed char / unsigned char	*int8_t* / *uint_8_t*	8-bit
signed int / unsigned int	*int16_t* / *uint16_t*	16-bit
signed long / unsigned long	*int32_t* / *uint32_t*	32-bit
signed long long / unsigned long long	*int64_t* / *uint64_t*	64-bit

Global Variables

- The use of global variables is not recommended. Use local variables whenever possible.
- If a variable is used only in a function, then it should be declared inside the function as a local variable.
- If a global variable is declared, a unique address in the SRAM will be assigned to this
  variable at program link time. Also accessing to a global variable will typically need extra bytes (usually two bytes for a 16 bits long address) to get its address.
- Example provided **1 Global Variable** increased code size by 20 bytes.

Loop Indices

- Use a decrement loop to optimize the comparison (*none needed for zero*)
- When looping, use pre-decrement `--i`, reduces compared code by 14 clock cycles
- Use `do {} while ()`, typically for optimal code

x86 Compiler Optimizations

- A compiler/assembly expert talks about compiler optimizations

Why xor eax, eax?

- Why xor eax, eax?
- Exclusive-OR a register with itself, zeros the register
- Therefore `xor eax, eax` (2 bytes)
- is better than `mov eax, 0x0` (5 bytes)
- AND the x86 CPU recognizes this *zeroing idiom* and optimizes it to zero bytes

Compiler Optimizations

Using the ISA differently

- Addressing the adding situation
- x86 is unusual in mostly having a maximum of two operands per instruction
- "*As someone who grew up with 6502, and then 32-bit ARM, coming to the x86 ISA was quite a shock. The x86 is truly a “Complex Instruction Set Computer”*".
- You can use `lea` (*load effective address*), with 3 operands and save the registers as well
- Example `lea eax, [rdi+rsi]`
- "*so lea becomes a cheeky way to do three-operand addition*"

Many, many more examples

Matt Godbolt's Blog: 2025 Advent of Compiler Optimisation

Reducing System Calls - Behind the Scenes of Bun Install

Bun

- an all-in-one toolkit for developing modern JavaScript/TypeScript applications (aka package manager and framework)
- Running `bun install` is fast, very fast. On average,
  - ~7× faster than npm
  - ~4× faster than pnpm
  - ~17× faster than yarn.
- Fast because it treats package installation as a systems programming problem
- In 2009, Node.js launched and
  - a typical disk seek takes 10ms
  - a database query 50–200ms
  - HTTP request to an external API 300ms+

Node.js is event-driven, like a browser

- Node.js figured that JavaScript's event loop (originally designed for browser events) was perfect for server I/O.
- When code makes an async request, the I/O happens in the background while the main thread immediately moves to the next task. Once complete, a callback gets queued for execution.

Its 2026 and

- Today's NVMe drives push 7,000 MB/s, 70× faster than what Node.js was designed for!
- Internet speeds stream 4K video and low-end smartphones have more RAM than high-end servers had in 2009.
- In other words, package management needed to improve
- Package managers still optimize for the last decade's problems. In 2025, the real bottleneck isn't I/O anymore. It's system calls.

Remember this?

```
add_2
; callee setup:
    ADD R6, R6, #-1           ; allocate spot for return value
    ADD R6, R6, #-1
    STR R7, R6, #0            ; push R7 (return address)
    ADD R6, R6, #-1
    STR R5, R6, #0            ; push R5 (caller frame pointer)
    ADD R5, R6, #-1           ; set frame pointer
...code...do something..
add_2_teardown
    ADD R6, R5, #1            ; pop local variables
    LDR R5, R6, #0            ; pop frame pointer
    ADD R6, R6, #1
    LDR R7, R6, #0            ; pop return address
    ADD R6, R6, #1
    RET
; end function
```

And this?...*Why Separate OS and User Code?*

The OS manages shared resources that every program depends on:

- **I/O devices** — keyboard, display, storage
- **Memory** — who owns which addresses
- **The CPU itself** — scheduling, exceptions, interrupts

Without separation, any program could:

- Overwrite OS code in memory — crashing the entire machine
- Read another program's data — violating privacy
- Drive hardware directly — causing device conflicts or data corruption
- Escalate its own privileges — bypassing all security

Every Computer Does It

- Every time your program wants the operating system to do something (read a file, open a network connection, allocate memory), it makes a system call. Each time you make a system call, the CPU has to perform a mode switch.
- **Your CPU can run programs in two modes:**
  - *user mode*, where your application code runs. Programs in user mode cannot directly access your device's hardware, physical memory addresses, etc.
  - *kernel mode*, where the operating system's kernel runs. The kernel is the core component of the OS that manages resources like scheduling processes to use the CPU, handling memory, and hardware like disks or network devices.

During a mode switch

- the CPU stops executing your program
- → saves all its state
- → switches into kernel mode
- → performs the operation
- → then switches back to user mode.

The cost of system calls

- a switch alone costs 1000-1500 CPU cycles in pure overhead (*3Ghz CPU, each cycle takes 0.33ns*)
- Installing React and its dependencies might trigger 50,000+ system calls

System Call Efficiency (`package install`)

Package	# syscalls	install time (s)
bun	166k	5.6
pnpm	457k	24.5
npm	997K	37.2
yarn	4047k	94.1

So what is the magic with Bun?

- npm, pnpm and yarn are all written in Node.js
- In Node.js, system calls aren’t made directly
- Node.js uses *libuv*, a C library (compiled)
- Many things must happen between Node.js and *libuv*, argument validation, string conversion, threading, kernal mode switching
- Bun - Written in Zig, compiled to native code with direct system call access

Net, Net:

Bun is more like a **native application** that happens to understand JavaScript packages, **not a JavaScript application** trying to do systems programming.

Additional Bun Optimizations

Many other optimizations as well, most typically *due to native ability, capabilities not native to JS*:

- binary manifests vs JSON manifests (binary comparisons vs. string comparisons)
- efficient use of memory to decompress tarballs (compressed packages)
- cache-friendly data layout
- native multi-core support (as compared to JS' multiple threads)

Python Package Management Background

- Python can be difficult to manage, due to versioning and interdependencies (*an extreme understatement!*)
- for years, its been *pip*, pythons package manager, typically `pip install xxx`
- And even in a fairly simple install, best practices:
  - install in an virtual environment (*venv*)
  - keep the *venv*'s separate
  - *pip* takes time, could be on the order of minutes

uv - a new Python package manager (Feb 2024)

- *"its written in Rust"*
- Much like Bun, it examined all of the activities which were happening
- Being compiled to native code, does make it fast (*pip is written in Python*)
- New standards allowed a clean slate approach as to *pip* having to maintain compatibility
- Rust has much better system-level tools to deal with hardware utilization
- *pip* downloads packages sequentially, while uv downloads many at once

Other Optimizations

Why CachyOS

"CachyOS provides a large selection of optimized packages specifically compiled for various modern CPU architectures. This includes support for x86-64-v3, x86-64-v4, and Zen4+ systems, ensuring your software is built to take full advantage of your hardware’s capabilities for a significant performance boost."

SIMD: Single Instruction, Multiple Data

Definition: One instruction executes the same operation on multiple data values simultaneously—parallelism at the data level.

How it works:

- Instead of: `ADD R1, R2, R3` (one pair of numbers)
- SIMD does: $\text{ADD} \begin{bmatrix} R1_a \\ R1_b \\ R1_c \\ R1_d \end{bmatrix}, \begin{bmatrix} R2_a \\ R2_b \\ R2_c \\ R2_d \end{bmatrix}$ (four pairs at once)

Real-world connection:

- Modern CPUs have **vector registers** (e.g., 128-bit, 256-bit) that hold multiple data items

Rust and SIMD: Parallel Processing in Software

Rust and SIMD

The Three-Step Pattern

All SIMD operations follow: **Load** → **Compute** → **Store**

- **Load:** Fill vector registers with data (e.g., 8 integers at once)
- **Compute:** Execute one instruction on all values (e.g., add 8 pairs simultaneously)
- **Store:** Write results back to memory

Real impact: AVX-512 implementations can achieve **10x speedups** with less than a day of work in Rust .

Three Levels of SIMD in Rust

Level	Description	Status
Auto-vectorization	LLVM compiler automatically converts loops to SIMD	Stable Rust
Portable SIMDs	Use generic types like `u32x8` (8 × 32-bit integers)	Nightly only
Raw Intrinsics	Direct assembly-level instructions like `_mm512_add_epi32`	Stable, platform-specific

Auto-Vectorization: Let the Compiler Work

For common operations, **write simple code and let LLVM optimize** :

```
// XOR two buffers - LLVM auto-vectorizes this
input_block
    .iter_mut()
    .zip(keystream)
    .for_each(|(p, k)| *p ^= k);
```

Rule: Don't manually optimize unless profiling proves it's a bottleneck

Portable SIMDs (Future-Focused)

Write once, compile for any platform with vector support

```
// u32x4 = 128-bit vector with 4 lanes (32-bit each)
let a = u32x4::splat(1);      // [1,1,1,1]
let b = u32x4::from([1,2,3,4]);
let result = a + b;           // [2,3,4,5]
```

- Rust chooses the correct instructions for x86, ARM, or WASM
- No duplicate code for different architectures

Raw Intrinsics: Maximum Control

Use the `std::arch` module for explicit hardware control on **stable Rust**:

- Direct mapping to CPU instructions: `_mm_add_epi32` (SSE2), `_mm256_add_epi32` (AVX2)
- Requires platform-specific code paths (x86_64 vs. aarch64)
- Most complex but guarantees specific instructions execute

Course connection: This is the closest to assembly programming—explicit register and instruction management.

Summary: When to Use What

1. **Default:** Write clean Rust—LLVM auto-vectorizes common patterns
2. **Hot paths needing guarantee:** Use portable `u32xN` types (when stabilized)
3. **Maximum performance, specific hardware:** Raw intrinsics with `std::arch`

turbopuffer

turbopuffer is a fast search engine that hosts 3.5T+ documents, handles 10M+ writes/s, and serves 25k+ queries/s. We are ready for far more. We hope you'll trust us with your queries

We reduced the latency on a full-text search query from 220ms → 47ms* by looking under the hood of Rust's "zero-cost" iterators to find that they were silently preventing vectorization. This post serves as a reminder that zero-cost abstractions do not absolve you from practicing mechanical sympathy.

Zero-Cost Abstractions: What They Really Cost

Key Concept: A "zero-cost" abstraction promises to compile into the same machine code you would write by hand—no extra instructions, no runtime overhead

- In C and higher-level languages, we use abstractions (functions, loops, iterators) to write cleaner code
- **The promise:** The compiler removes the abstraction completely during optimization
- **Example:** Rust's `Iterator` trait claims to generate code identical to a hand-written `for` loop

The Hidden "Opportunity Cost"

Zero-cost does **not** mean zero impact on performance.

The Problem: While each individual call compiles efficiently, the abstraction creates boundaries that hide the loop structure from the compiler

- **SIMD (Single Instruction, Multiple Data):** Modern CPUs can process 4, 8, or 16 data items in parallel using special vector registers
- **Loop Unrolling:** The compiler replicates loop bodies to reduce branch overhead

Why it fails: The compiler cannot see across abstraction boundaries to apply these optimizations across multiple iterations

Looking inside the merge iterator

If we scan 100,000 integers on a modern 3 GHz CPU, we would expect it to take ~130µs (100K values × 4 instructions each ÷ 3 billion cycles/sec), assuming only scalar operations. In reality, it takes 6.5ms! 50× more expensive than it should be if the iterator is truly "zero-cost".

At first glance, nothing in Rust explains the cost. The work is occurring purely in memory, so we can eliminate I/O from consideration, yet something is clearly preventing the CPU from ripping through these integers. Every engineer knows that computers are a tower of ~~lies~~ abstractions, so we need to go below the abstraction to look at what the compiler is actually generating.

The cost hides beneath the abstraction

"*Herein lies the hidden opportunity cost of Rust's zero-cost abstraction in our merge iterator. The iterator itself compiles down to the code you'd write by hand for a single call. In that sense, it is zero-cost. But the abstraction also prevents the compiler from vectorizing or unrolling across calls, so even though the loop body is exactly the kind of dumb and serial work modern CPUs devour, the compiler can't vectorize it. And the recursive calls to next() bury 130µs of useful work inside 6.5ms of merge overhead.*"

Summary: Lessons for Programmers

1. **"Zero-cost" refers to single operations, not whole programs**
2. **Abstractions can prevent global optimizations** like SIMD and unrolling across calls
3. **Know your target hardware:** Modern CPUs have vector units that require predictable data access patterns
4. **Sometimes break the abstraction:** When performance matters, expose the full loop structure to the compiler

Bottom line: Write clean code first, but understand the assembly-level consequences when performance is critical.

CPU Stagnant?

Three processors with comparable prices at release:

Year	Processor	json parsing speed
2024	AMD Ryzen 7 9800X3D (Zen 5, with up to 5.3 GHz boost)	12.7GB/s
2023	AMD Ryzen 7 7800X3D (Zen 4, with up to 5.1 GHz boost)	9 GB/s
2022	AMD Ryzen 7 5800X3D (Zen 3, with 3.4 GHz base)	5.2 GB/s

"*In the case of this benchmark, the simdjson library is designed to benefit from better processor features.*"

simdjson

"*JSON is everywhere on the Internet. Servers spend a lot of time parsing it. We need a fresh approach.*"

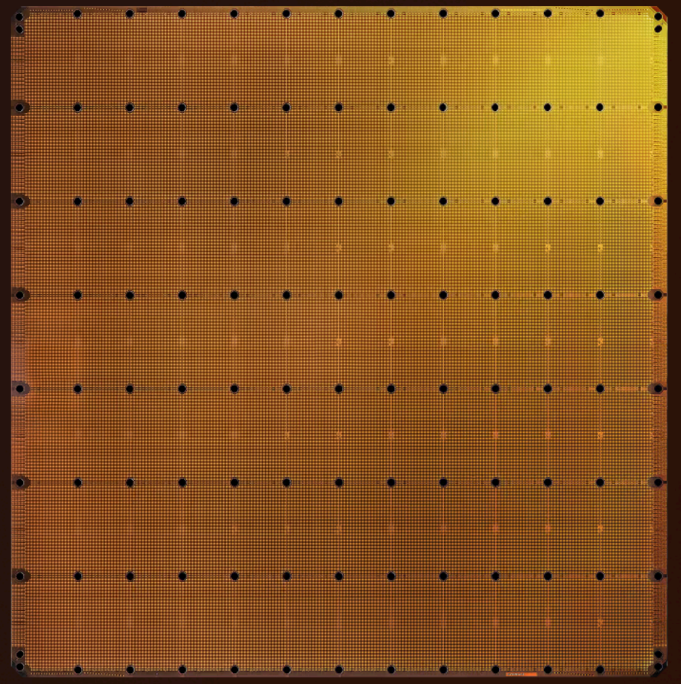
- library automatically detects the features supported by the processor.
- runs on all 64-bit systems, but it automagically benefits from advanced processors.
- Broad hardware support: Optimized kernels for x64 (including AVX-512), 64-bit ARM, RISC-V, Loongarch, and so forth.

THE FUTURE OF AI IS WAFER SCALE

Four trillion transistors. 125 petaflops. One silicon wafer. The world's largest and most powerful processor for AI training and inference.

READ THE WHITEPAPER

bottom



Cerebras
Wafer Scale Engine 3

Nvidia GPU
B200

Cerebras AI Chip

- The WSE-3 is the largest AI chip ever built, measuring 46,225 mm² and containing 4 trillion transistors. It delivers 125 petaflops of AI compute through 900,000 AI-optimized cores — 19× more transistors and 28× more compute than the NVIDIA B200.
- With 44 GB of on-chip SRAM and 21 petabytes per second of memory bandwidth, the WSE-3 eliminates traditional memory bottlenecks. Its wafer-scale fabric delivers 27 petabytes per second of internal bandwidth — 206× faster than the latest generation of NVLink.
- The WSE-3 powers Cerebras Inference — the world’s inference API. At over 3,000 tokens/s on OpenAI's GPT OSS 120B, it is 15× faster than leading GPU cloud. High speed inference unlocks real-time reasoning, instant code gen, and agentic apps that are simply impossible on GPUs.